

Thresholding

Thresholding

Preface

Example Results

Principle Explanation

Global Threshold

OTSU Automatic Threshold

Adaptive Threshold

Code Overview

Initialize Camera and Display Module

Set Threshold Mode

Convert to Grayscale

Fixed Threshold and OTSU

Adaptive Threshold

Display and Resource Release

Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the Jianan website https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/



Example Results

Run the example code for this section [12.camera_threshold.py](#).

This section converts the live image to grayscale and demonstrates fixed global threshold, OTSU automatic threshold, adaptive Gaussian threshold, and adaptive mean inverted threshold.





Principle Explanation

Global Threshold

Global threshold uses the same cutoff value for the entire image. Taking binary threshold as an example:

```
Pixel value > threshold → 255  
Pixel value ≤ threshold → 0
```

Fixed threshold is simple and fast, but when illumination is uneven, bright and dark regions of the same target may be separated.

OTSU Automatic Threshold

OTSU selects the threshold that maximizes inter-class variance based on the grayscale histogram, suitable for scenes where foreground and background form two distinct grayscale peaks. It is still a global threshold and cannot solve severe local shadows.

Adaptive Threshold

Adaptive threshold computes a local neighborhood threshold for each pixel, more suitable for uneven illumination. The Gaussian method weights the neighborhood, while the mean method uses equal-weight averaging.

Code Overview

Initialize Camera and Display Module

```
W, H = 640, 480  
sensor = Sensor(width=1280, height=960, fps=90)  
sensor.reset()  
sensor.set_framesize(width=W, height=H)  
sensor.set_pixformat(Sensor.RGB888)  
Display.init(Display.ST7701, width=640, height=480, to_ide=True)  
sensor.run()  
time.sleep(0.5)
```

Set Threshold Mode

```
mode = 1
```

mode	Method	Parameters
0	Fixed Binary Threshold	threshold 127, max value 255
1	OTSU	initial threshold 0, source default
2	Adaptive Gaussian	block size 11, C=2
3	Adaptive Mean Inverted	block size 15, C=3

Convert to Grayscale

```
gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)
```

The threshold function input is a single-channel 8-bit image.

Fixed Threshold and OTSU

```
if mode == 0:
    ret, out = cv2.threshold(
        gray, 127, 255,
        cv2.THRESH_BINARY
    )
elif mode == 1:
    ret, out = cv2.threshold(
        gray, 0, 255,
        cv2.THRESH_OTSU
    )
```

ret saves the actual threshold. In fixed mode it is usually 127; in OTSU mode it is the value computed from the current frame.

Adaptive Threshold

```
elif mode == 2:
    out = cv2.adaptiveThreshold(
        gray, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        11, 2
    )
else:
    out = cv2.adaptiveThreshold(
        gray, 255,
        cv2.ADAPTIVE_THRESH_MEAN_C,
        cv2.THRESH_BINARY_INV,
        15, 3
    )
```

```
)
```

Block size must be an odd number greater than 1. C is subtracted from the local statistic value to fine-tune the foreground range. Mode 3 uses inverted output, so dark targets become white foreground.

Display and Resource Release

```
img_out = image.Image(  
    w, h, image.GRAYSCALE,  
    alloc=image.ALLOC_REF,  
    data=out  
)  
Display.show_image(img_out)  
  
finally:  
    sensor.stop()  
    Display.deinit()  
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)  
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Fixed Threshold 127:** First observe the pixel range of target and background in the grayscale image, then select a middle value.
- **OTSU Usage Conditions:** When the scene grayscale distribution does not have an obvious bimodal pattern, the automatic threshold may not be reliable.
- **Block Size:** Larger is closer to slowly changing local background; smaller adapts better to fine brightness changes but is more prone to noise.
- **Constant C:** Increasing or decreasing changes the white foreground proportion. It should be adjusted separately after fixing illumination and block size.
- **Foreground Polarity:** Contours and morphology typically work with white foreground. Choose THRESH_BINARY or THRESH_BINARY_INV based on whether the target is bright or dark.
- **Preprocessing:** When there is a lot of binary noise, you can apply small-kernel smoothing before thresholding, or perform opening and closing operations after thresholding.